

Advanced Machine Learning

Breast Cancer Dataset – ML Algorithm Comparison

Complete Code + Program Output

```
# =====
# ML ALGORITHM COMPARISON
# Dataset: Breast Cancer Dataset
# Algorithms: Decision Tree, KNN, Gaussian Naive Bayes
# =====

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.metrics import f1_score, confusion_matrix, ConfusionMatrixDisplay

data = load_breast_cancer()
X = data.data
y = data.target

print("Dataset Shape:", X.shape)
print("Classes:", data.target_names)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Gaussian Naive Bayes": GaussianNB()
}

results = []

for name, model in models.items():
    if name == "Decision Tree":
        model.fit(X_train, y_train)
        prediction = model.predict(X_test)
    else:
        model.fit(X_train_scaled, y_train)
        prediction = model.predict(X_test_scaled)

    results.append([
        name,
        accuracy_score(y_test, prediction),
        precision_score(y_test, prediction),
        recall_score(y_test, prediction),
        f1_score(y_test, prediction)
    ])

results = pd.DataFrame(
    results,
    columns=["Algorithm", "Accuracy", "Precision", "Recall", "F1-Score"]
)

print("\n===== ALGORITHM COMPARISON =====")
print(results.round(4).to_string(index=False))

best = results.loc[results["F1-Score"].idxmax()]

print("\n===== BEST ALGORITHM =====")
print("Algorithm:", best["Algorithm"])
print("F1-Score:", round(best["F1-Score"], 4))

results.set_index("Algorithm").plot(
    kind="bar",
    figsize=(8, 5)
)

plt.title("ML Algorithm Performance")
plt.ylabel("Score")
plt.ylim(0, 1.05)
plt.xticks(rotation=0)
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

for name, model in models.items():
    if name == "Decision Tree":
        model.fit(X_train, y_train)
        prediction = model.predict(X_test)
    else:
        model.fit(X_train_scaled, y_train)
        prediction = model.predict(X_test_scaled)
```

```
cm = confusion_matrix(y_test, prediction)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=data.target_names
)
disp.plot()
plt.title(name + " - Confusion Matrix")
plt.tight_layout()
plt.show()
```

PROGRAM OUTPUT

Dataset Shape: (569, 30)

Classes: [np.str_('malignant'), np.str_('benign')]

===== ALGORITHM COMPARISON =====

Algorithm	Accuracy	Precision	Recall	F1-Score
Decision Tree	0.9123	0.9559	0.9028	0.9286
KNN	0.9561	0.9589	0.9722	0.9655
Gaussian Naive Bayes	0.9298	0.9444	0.9444	0.9444

===== BEST ALGORITHM =====

Algorithm: KNN

F1-Score: 0.9655

===== CONFUSION MATRICES =====

Decision Tree

	Predicted 0	Predicted 1
Actual 0	39	3
Actual 1	7	65

KNN

	Predicted 0	Predicted 1
Actual 0	39	3
Actual 1	2	70

Gaussian Naive Bayes

	Predicted 0	Predicted 1
Actual 0	38	4
Actual 1	4	68